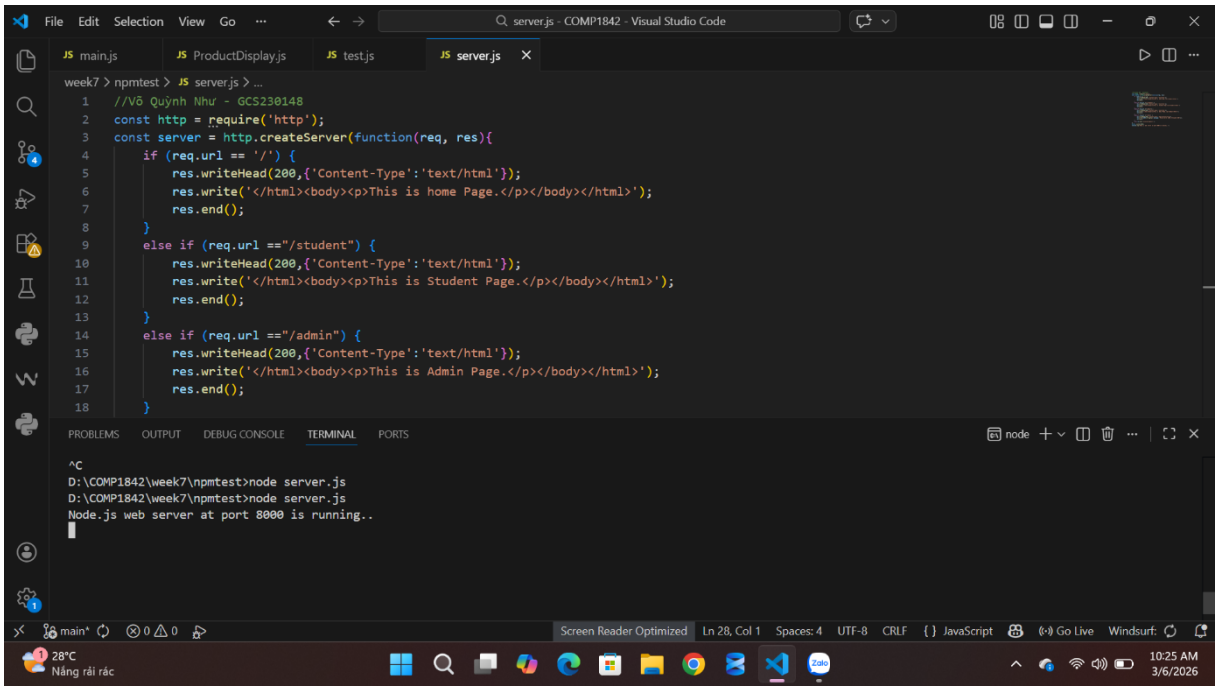
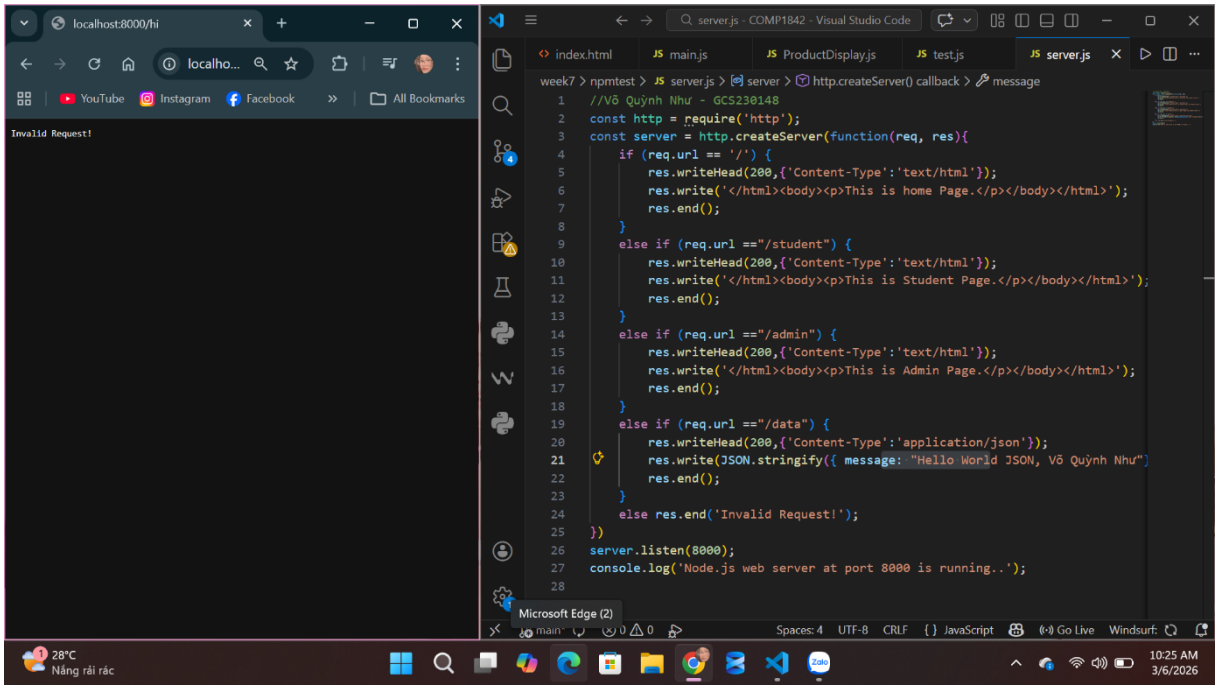
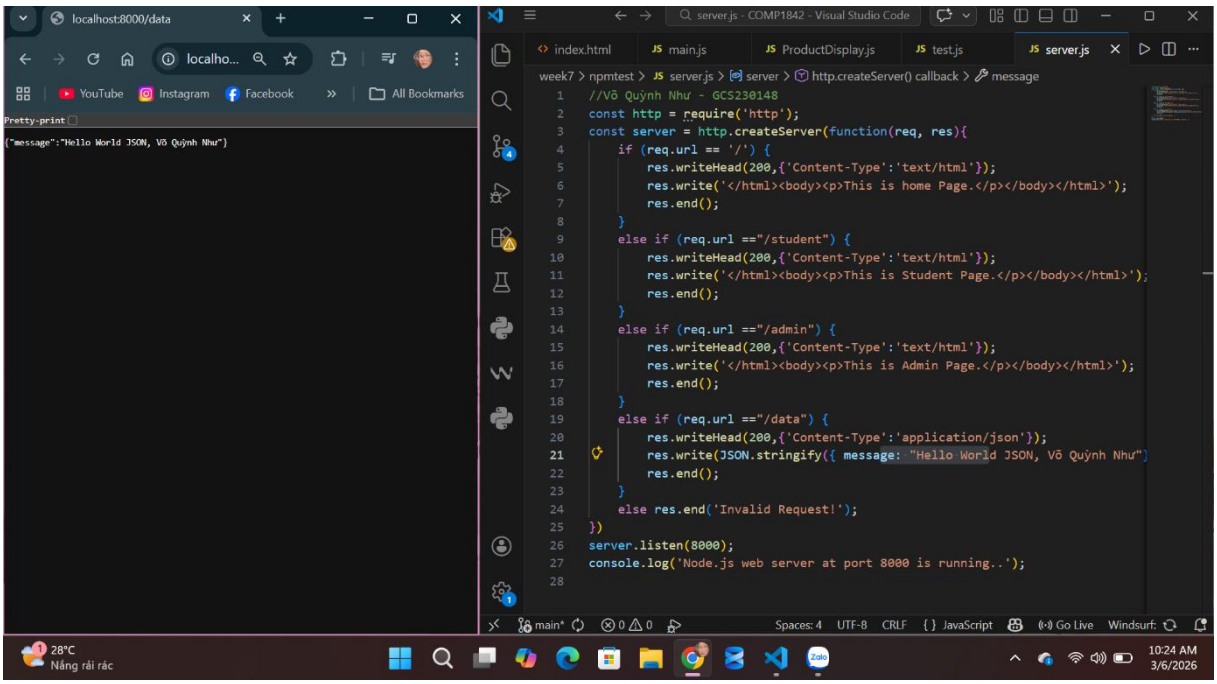
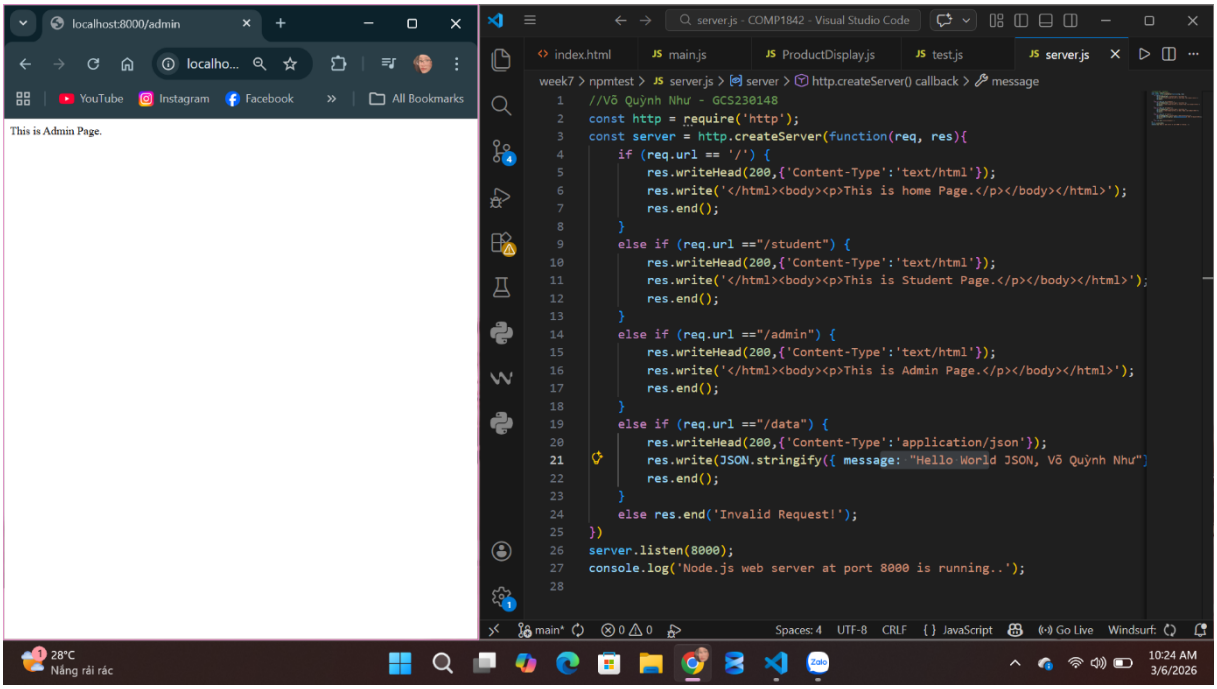
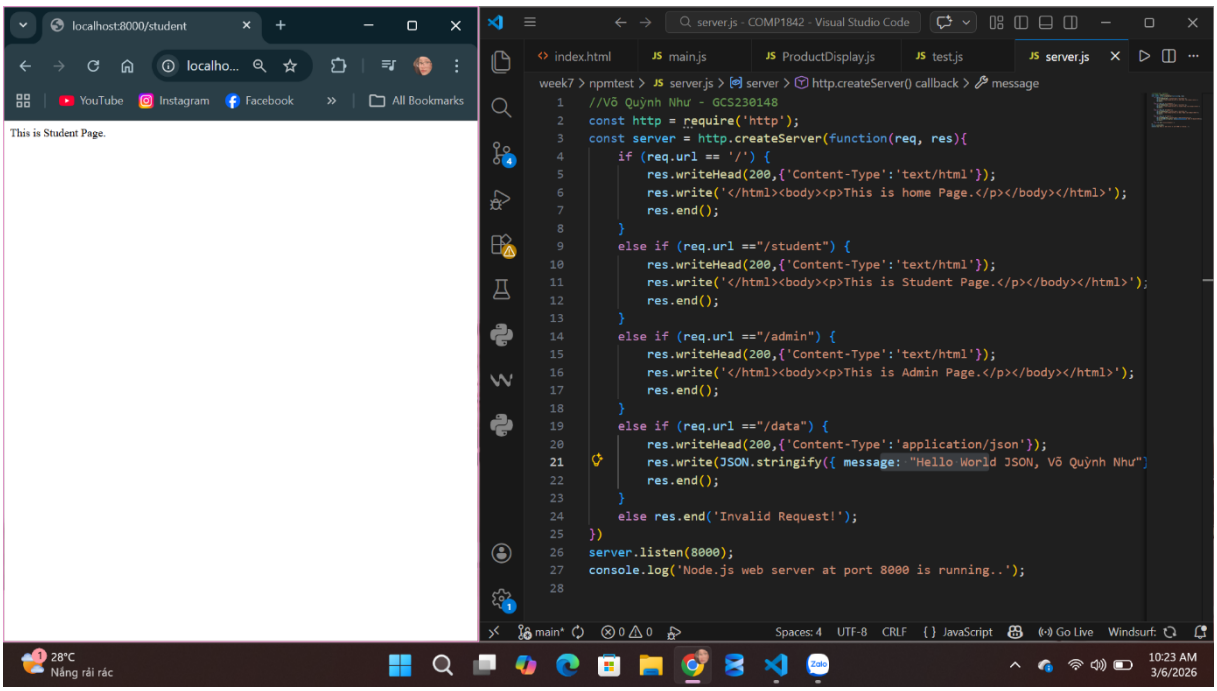
WEEK7

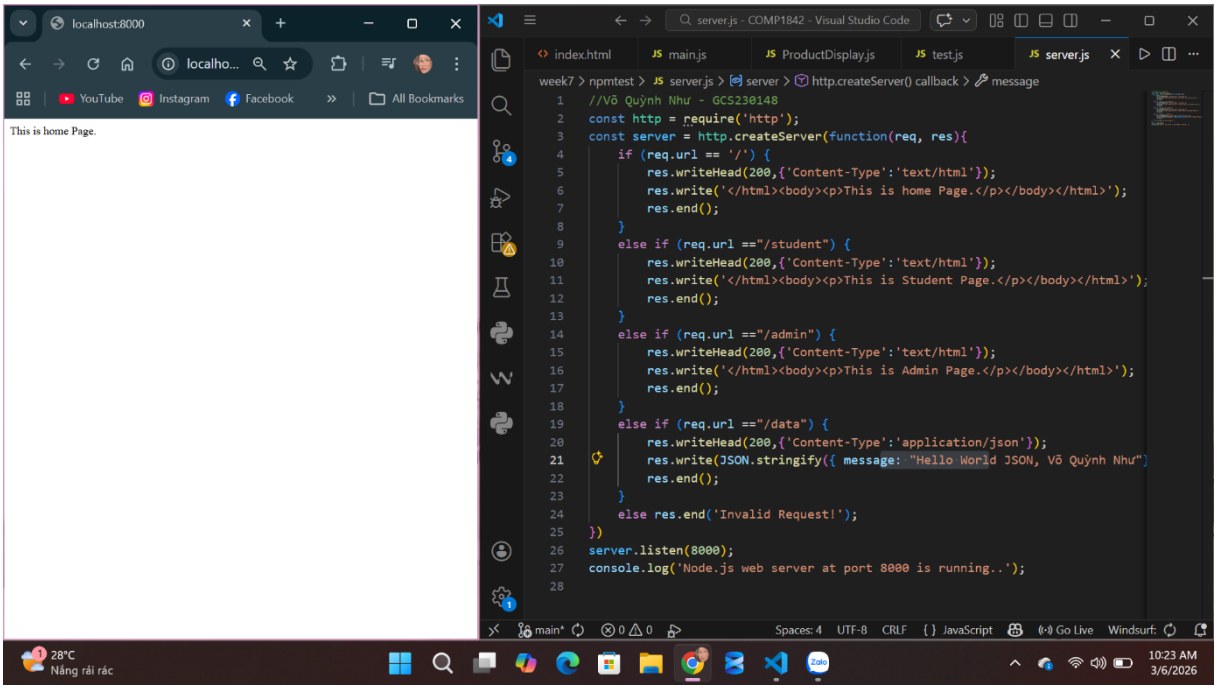

server.js - COMP1842 - Visual Studio Code
main.js
ProductDisplay.js
test.js
server.js
week7 > npmtest > server.js > ...
//Võ Quỳnh Như - GCS230148
const http = require('http');
const server = http.createServer(function(req, res){
if (req.url == '/') {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is home Page. </body></html>');
res.end();
}
else if (req.url =="/student") {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is Student Page. </body></html>');
res.end();
}
else if (req.url =="/admin") {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is Admin Page. </body></html>');
res.end();
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
^C
D:\COMP1842\week7\npmtest>node server.js
D:\COMP1842\week7\npmtest>node server.js
Node.js web server at port 8000 is running..


localhost:8000/hi
Invalid Request!
server.js - COMP1842 - Visual Studio Code
index.html
main.js
ProductDisplay.js
test.js
server.js
week7 > npmtest > server.js > server > http.createServer() callback > message
//Võ Quỳnh Như - GCS230148
const http = require('http');
const server = http.createServer(function(req, res){
if (req.url == '/') {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is home Page. </body></html>');
res.end();
}
else if (req.url =="/student") {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is Student Page. </body></html>');
res.end();
}
else if (req.url =="/admin") {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is Admin Page. </body></html>');
res.end();
}
else if (req.url =="/data") {
res.writeHead(200,{'Content-Type':'application/json'});
res.write(JSON.stringify({ message: "Hello World JSON, Võ Quỳnh Như"})
res.end();
}
else res.end('Invalid Request!');
})
server.listen(8000);
console.log('Node.js web server at port 8000 is running..');


localhost:8000/data
Pretty-print
{"message":"Hello World JSON, Võ Quỳnh Như"}
server.js - COMP1842 - Visual Studio Code
week7 > npmtest > server.js > server > http.createServer() callback > message
//Võ Quỳnh Như - GCS230148
const http = require('http');
const server = http.createServer(function(req, res){
if (req.url == '/') {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is home Page. </body></html>');
res.end();
}
else if (req.url =="/student") {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is Student Page. </body></html>');
res.end();
}
else if (req.url =="/admin") {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is Admin Page. </body></html>');
res.end();
}
else if (req.url =="/data") {
res.writeHead(200,{'Content-Type':'application/json'});
res.write(JSON.stringify({ message: "Hello World JSON, Võ Quỳnh Như"
res.end();
}
else res.end('Invalid Request!');
})
server.listen(8000);
console.log('Node.js web server at port 8000 is running..');


localhost:8000/admin
This is Admin Page.
server.js - COMP1842 - Visual Studio Code
week7 > npmtest > server.js > server > http.createServer() callback > message
//Võ Quỳnh Như - GCS230148
const http = require('http');
const server = http.createServer(function(req, res){
if (req.url == '/') {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is home Page. </body></html>');
res.end();
}
else if (req.url =="/student") {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is Student Page. </body></html>');
res.end();
}
else if (req.url =="/admin") {
res.writeHead(200,{'Content-Type':'text/html'});
res.write('</html><body> This is Admin Page. </body></html>');
res.end();
}
else if (req.url =="/data") {
res.writeHead(200,{'Content-Type':'application/json'});
res.write(JSON.stringify({ message: "Hello World JSON, Võ Quỳnh Như"
res.end();
}
else res.end('Invalid Request!');
})
server.listen(8000);
console.log('Node.js web server at port 8000 is running..');





Understanding

During Week 7, we learned how to build a simple web server using Node.js and how server-side JavaScript can respond to different requests from a web browser. Through this lecture and practice, we understood that Node.js allows JavaScript to run outside the browser and interact with system resources to create backend services. One important concept we learned is the use of the HTTP module, which allows us to create a web server by using the http.createServer() function. This function listens for incoming requests and sends responses back to the client.

In our practice, we created a simple server that runs on port 8000 and responds to different URLs. By checking the value of req.url, we were able to control which content the server returns. For example, when users access /, the server returns a Home Page

message. When users visit /student or /admin, the server responds with different HTML content that displays the corresponding page. This helped us understand the basic idea of routing, where the server determines what response should be sent depending on the requested path.

Another important thing we learned is how to send different types of responses. For HTML pages, we used the Content-Type: text/html, while for data responses we used Content-Type: application/json. This allowed us to send structured data in JSON format, such as the message displayed when accessing /data. We also saw how the server handles invalid requests by returning a simple message like “Invalid Request”.

Through this lecture, we gained a clearer understanding of how a basic web server works, how browsers communicate with servers using HTTP requests, and how Node.js can be used to handle those requests and return appropriate responses. This knowledge forms the foundation for building more advanced backend applications in the future.